FUELBOOK

ABSTRACT

FuelBook is an Android-based digital fuel management and reporting system developed for fuel station operations. Its main purpose is to replace manual register-based fuel accounting with a faster and more reliable software solution. In many fuel stations, daily sales and payment records are still maintained on paper. This manual process is time-consuming, repetitive, and prone to calculation errors, making it difficult to verify daily collections and prepare accurate reports.

FuelBook addresses this problem by digitizing fuel entry, payment collection, and daily report generation. The application allows operators to record petrol and diesel sales using nozzle opening and closing readings. It automatically calculates litres sold and the corresponding sale amount based on the entered fuel price, reducing human effort in meter-based calculations. The system also supports cash collection entry using currency denomination counts.

In addition, it records digital payment collections through multiple UPI platforms such as Paytm, Google Pay, and PhonePe. By combining fuel sales data and payment collections, the system provides a complete daily operational view. A daily summary module aggregates petrol sales, diesel sales, cash total, UPI total, and overall collection. It also calculates the difference between fuel sale value and collected payment amount, helping identify mismatches, shortages, or excess collections.

The application stores all operational data in a local Room database built on SQLite, ensuring structured, reliable, and persistent storage. FuelBook also maintains report history so past daily reports can be viewed, filtered, and managed easily. The project provides a simple mobile interface with modules for fuel entry, cash collection, UPI collection, summary, and history, along with settings for fuel prices and pump preferences.

A PDF generation feature enables downloading and sharing professional daily reports, useful for record keeping, auditing, and communication with management. Technically, the project is implemented using Kotlin, Android SDK, Material Design components, coroutines, and Room database architecture, following a modular structure.

The final outcome is a practical mobile application that simplifies fuel station bookkeeping and reporting, improving accuracy, transparency, and daily decision-making through automation.

INTRODUCTION

Fuel management is an important part of transport, logistics, and fuel station operations. Fuel is a major operating expense for vehicles, generators, and commercial transport systems. In petrol pumps and fleet operations, daily fuel sales and collections must be recorded accurately, as even small errors can affect profit, stock monitoring, and auditing. Because of this, fuel data must be managed in a systematic and reliable way.

In many small and medium fuel stations, record keeping is still done manually. Operators write nozzle readings, fuel prices, and payment details in notebooks or registers. Cash collections are counted by hand, and UPI transactions are noted separately. At the end of the day, staff manually calculate total sales and compare them with collections. While simple, this method creates practical problems.

Manual systems are time-consuming and depend on human accuracy. Repeated calculations increase the chance of errors. Paper records can be lost, damaged, or difficult to verify later, and searching old records is slow. Manual reporting also makes it difficult to identify mismatches between fuel sold and money collected. The lack of real-time summaries and structured reporting further complicates reconciliation, reducing transparency and efficiency.

FuelBook is designed to solve these issues through a mobile-based fuel management system. It is an Android application that digitizes fuel entry, cash collection, UPI collection, and report generation. The system allows users to enter nozzle readings for petrol and diesel and automatically calculates litres sold and corresponding amounts based on fuel price, reducing manual work and improving consistency.

FuelBook records cash collections using denominations and supports UPI entries through Paytm, Google Pay, and PhonePe. All data is stored in a structured local database using Room and SQLite. A daily summary screen combines fuel sales and collections into a clear report and calculates differences for reconciliation.

The system includes report history and a PDF export feature for sharing and record keeping, along with settings for fuel prices and preferences. By digitizing processes, FuelBook improves speed, accuracy, and traceability, helping staff complete daily reporting efficiently and providing a practical solution for modern fuel management.

PROBLEM STATEMENT

Fuel management is a daily operational activity in fuel stations and transport-related businesses. It involves recording fuel sales, monitoring payment collections, and preparing accurate daily reports. In many places, these activities are still handled manually using paper registers or basic calculators, which are not efficient for fast and error-free operations. As the number of transactions increases, manual handling becomes difficult.

One major issue in existing systems is the dependence on handwritten records. Operators must note nozzle readings, fuel quantities, and payment details separately, increasing the chances of missing entries, duplicate records, or incorrect calculations. Even a small error can make the final daily report inaccurate, affecting accountability and financial transparency.

Another problem is the separation of fuel and payment records. Fuel sales may be recorded in one register, while cash and UPI collections are recorded elsewhere. At the end of the day, staff must compare these values manually, which is slow and confusing. It becomes difficult to identify whether mismatches are due to fuel entry errors or payment counting errors.

Manual systems also make report generation difficult. Preparing daily sales reports requires repeated calculations and checking multiple records, consuming valuable time. In busy petrol pumps, staff need faster ways to generate summaries and maintain consistency. Without automation, daily closing activities become stressful and inefficient.

Another issue is poor historical tracking. Paper records are difficult to search, filter, or analyze. Checking past reports takes time, and reviewing long-term sales trends or payment patterns becomes challenging, limiting informed decision-making.

The existing approach also lacks proper digital storage and sharing. There is no easy way to generate professional reports for management or audits. As digital payments increase, manual systems become less suitable.

Modern fuel operations require a system that integrates fuel entries, cash collections, and UPI payments in one place, with instant calculations and secure storage. FuelBook is developed to solve this problem by providing a simple, reliable, and digital fuel management solution.

OBJECTIVES

- To develop a mobile-based fuel management system for daily operational use.
- To replace manual fuel record keeping with a digital and structured solution.
- To record petrol and diesel sales using nozzle opening and closing readings.
- To automatically calculate fuel quantity sold in litres.
- To compute sale amount based on entered fuel price and recorded readings.
- To provide separate modules for fuel entry, cash collection, and UPI collection.
- To simplify daily reconciliation between fuel sales and money collected.
- To record cash transactions using denomination-wise input for better accuracy.
- To support digital payment tracking through UPI platforms such as Paytm, Google Pay, and PhonePe.
- To generate a daily summary showing total petrol sales, diesel sales, cash total, and UPI total.
- To calculate the difference between total fuel sale and total payment collection.
- To help users quickly detect shortages, excess collection, or accounting mismatch.
- To store data securely in a local database for reliable access and persistence.
- To maintain historical records of daily reports for future review.
- To allow users to search and view reports by date range.
- To provide PDF report generation for easy sharing, printing, and documentation.
- To improve the speed of end-of-day reporting activities at fuel stations.
- To reduce arithmetic mistakes caused by manual calculations.
- To minimize paperwork and repetitive register-based entry processes.
- To improve transparency in daily fuel and payment records.
- To support better monitoring of operational performance over time.
- To provide a simple and user-friendly interface that can be used by pump staff easily.
- To ensure the application remains practical for real-world petrol pump environments.

LITERATURE REVIEW

Fuel management has been handled through manual registers, spreadsheets, fleet tools, and mobile expense applications. Traditional paper-based systems are still common in small fuel stations and local transport operations. These rely on handwritten entries, manual calculations, and physical storage. While simple, they become inefficient as transactions increase and often lead to delayed reporting and human error.

Spreadsheet-based solutions like Excel allow users to organize fuel quantity, price, and expenses in tabular form. They improve upon paper systems with formula-based calculations and digital storage. However, they still require manual data entry and file management, making them less suitable for real-time, field-level, or mobile-first fuel station operations.

Many mobile applications focus on personal vehicle fuel tracking. These apps help users monitor mileage, fuel refill history, cost, and consumption. While useful for individuals, they do not support petrol pump workflows such as nozzle reading entry, shift collection recording, and day-end reconciliation. This creates a gap between personal tracking tools and commercial fuel operation needs.

Fleet management systems provide advanced features like GPS tracking, route optimization, driver monitoring, fuel analysis, and maintenance alerts. They are effective for large logistics organizations but are often costly, complex, and dependent on external hardware or cloud systems, making them unsuitable for small fuel stations.

Accounting and billing software also support transaction management, report generation, and historical data tracking. However, they do not focus on fuel-specific operations such as nozzle readings, litres sold, or fuel-type summaries. Users still need separate systems for fuel recording and financial tracking.

A major limitation across these solutions is the lack of integration. Most tools address only one aspect, such as expenses, payments, or mileage. Few combine fuel entry, cash handling, UPI collection, daily summaries, report history, and PDF export in a single platform. This separation increases effort and reduces efficiency.

FuelBook addresses these gaps by offering an integrated fuel management system. It combines petrol and diesel entry, cash collection, UPI recording, daily summaries, and report storage. With automatic calculations and PDF export, it provides a practical, low-cost solution for efficient fuel management.

METHODOLOGY

FuelBook is a mobile-based fuel management application developed using Android and Kotlin. It follows a modular methodology where each operational task is handled through separate but connected components. Its workflow is based on data flow from user input to processing, storage, aggregation, and report generation, improving maintainability, accuracy, and usability.

The system begins with the user interface, where operators enter daily fuel and payment details. Frontend screens are built using Android XML layouts and Material Design components, providing modules for fuel entry, cash collection, UPI collection, daily summary, history, and settings. This modular design keeps the workflow simple and organized.

In the fuel entry module, users record petrol and diesel nozzle readings, including opening, closing, and fuel price. The system automatically calculates litres sold and total amount, reducing manual effort and improving consistency. This process is repeated for multiple nozzles to generate totals.

In the cash collection module, users enter counts of currency notes and coins. The system multiplies counts with denominations to calculate total cash, ensuring accuracy. The UPI module records digital payments from different providers and computes the total automatically.

After data entry, backend processing is handled through repository logic using Kotlin classes and coroutines. The repository connects frontend activities with the Room database, validating and storing fuel, cash, and UPI data, ensuring clean interaction between layers.

The database layer uses Room on SQLite with separate tables for fuel entries, cash, UPI data, and report history. Each record includes attributes like date, transaction values, and timestamps, enabling structured storage and retrieval.

The daily summary module aggregates stored data to calculate petrol and diesel totals, cash, UPI, and overall collection. It also computes the difference between fuel sales and collections for reconciliation. Results are displayed and can be saved as reports or exported as PDFs.

The history module allows users to view, filter, delete, and regenerate reports. Shared preferences handle lightweight settings like fuel prices. Thus, FuelBook follows a structured methodology of input, processing, storage, and output for efficient fuel management.

SYSTEM REQUIREMENTS

A. Hardware Requirements

1. Processor: Intel Core i3 or above
2. RAM: Minimum 4 GB, recommended 8 GB
3. Storage: Minimum 2 GB free space for development tools and project files
4. Display: Standard monitor with 1366 x 768 resolution or higher
5. Input Devices: Keyboard and mouse
6. Android Device: Smartphone or emulator for testing
7. Mobile RAM: Minimum 2 GB for smooth application execution
8. Mobile Storage: Minimum 200 MB free space
9. Internet: Required for downloading dependencies during development
10. Power Backup: Recommended to avoid interruption during build and testing

B. Software Requirements

1. Operating System: Windows 10/11, Linux, or macOS
2. Development Environment: Android Studio
3. Programming Language: Kotlin
4. Markup/Layout Language: XML
5. Build System: Gradle with Kotlin DSL
6. Database: SQLite through Room persistence library
7. Minimum Android SDK: API level 24
8. Target Android SDK: API level 36
9. Java Version: Java 11
10. Version Control: Git
11. Testing Environment: Android Emulator or physical Android device

C. Packages/Tools Used

1. Android SDK: Core platform for Android application development
2. Android Studio: IDE used for coding, debugging, building, and testing
3. Kotlin: Main programming language used to implement the application
4. XML Layout System: Used for designing frontend screens
5. Gradle: Used for dependency management and project build automation
6. Room Database: Used for structured local data storage
7. SQLite: Underlying database engine used by Room
8. Kotlin Coroutines: Used for asynchronous background operations
9. AndroidX Core KTX: Used for Kotlin-friendly Android APIs
10. AndroidX AppCompat: Used for backward compatibility and UI support
11. Material Design Components: Used to create modern user interfaces
12. AndroidX Lifecycle ViewModel KTX: Used for lifecycle-aware architecture support
13. AndroidX Lifecycle LiveData KTX: Used for observable UI-related data handling
14. Data Binding: Used for connecting UI components with data
15. FileProvider: Used for secure PDF file sharing
16. PdfDocument API: Used for report generation in PDF format

D. Optional Requirements

1. Robolectric: Used for local Android unit testing
2. JUnit: Used for writing and running test cases
3. AndroidX Test Core: Used for Android testing support
4. AndroidX Test Ext JUnit: Used for Android-based unit and instrumentation testing
5. Espresso Core: Used for UI testing
6. Backup and data extraction rules: Used for application data handling support
7. SharedPreferences: Used for lightweight local storage of settings and user session data
8. Physical printer or PDF viewer: Optional for printed report usage
9. Cloud synchronization: Not currently implemented, but can be added in future versions
10. Authentication module: Basic session handling exists; full login system can be extended later.

- **ADVANTAGES**

1. Reduces manual paperwork in daily fuel and payment management.
2. Improves accuracy by automatically calculating litres sold and sale amount.
3. Combines fuel entry, cash collection, and UPI collection in one application.
4. Saves time during end-of-day reconciliation and report preparation.
5. Helps identify differences between fuel sales and total collections quickly.
6. Stores records digitally, which improves safety and accessibility over paper registers.
7. Makes historical reports easier to search, review, and manage.
8. Supports PDF generation for reporting, sharing, and documentation purposes.
9. Provides a user-friendly mobile interface suitable for daily operational use.
10. Reduces repetitive calculation errors in cash denomination counting.

- **DISADVANTAGES**

1. The current system uses local storage only, so data is limited to a single device unless manually shared.
2. A complete login and role-based authentication system is not fully implemented.
3. The application is currently focused on Android devices and does not support other platforms directly.
4. Some settings features, such as custom nozzle and UPI option persistence, are only partially implemented.
5. The current project uses fixed test dates in some modules, which limits real-time production readiness.
6. PDF export depends on device storage permissions and Android file handling behavior.

CONCLUSION

FuelBook is a practical digital solution developed to improve fuel management and daily reporting activities. It replaces manual record keeping with a mobile-based application and provides a structured method to record fuel sales, cash collections, UPI payments, and summary reports. This reduces dependency on paper registers and manual calculations, improving speed and accuracy in routine operations.

The project achieves its core objective of creating an integrated fuel management platform with modules for fuel entry, cash collection, UPI collection, daily summary, report history, and settings. These modules work together to support the complete day-end workflow of a fuel station. The system automatically calculates litres sold, fuel amount, total collection, and collection difference, reducing human error in arithmetic and reconciliation.

FuelBook demonstrates effective use of Android technologies, implemented using Kotlin, XML, Room database, SQLite, coroutines, and Material Design components. Data is stored in a structured and persistent manner, improving reliability. Historical reports can be viewed, filtered by date, and exported as PDF documents, making the system useful for daily operations, audits, and management.

From a real-world perspective, the project has strong practical value. Many petrol pumps still depend on manual systems that are slow, repetitive, and difficult to verify. FuelBook offers a better alternative by digitizing the reporting process in a simple mobile interface, helping operators perform daily tasks more efficiently and confidently.

The system improves transparency, accountability, and data organization. By combining fuel entries and payment collections in one place, it simplifies reconciliation and supports better record maintenance compared to paper-based methods. This helps business owners monitor performance effectively over time.

The project also has future scope for enhancement, such as adding authentication with user roles, cloud backup and synchronization, advanced dashboards, analytics, and chart-based insights. Additional features like vehicle-wise mileage tracking, stock monitoring, and predictive fuel analysis can further expand its usefulness.

In conclusion, FuelBook delivers a reliable and scalable fuel management application, addressing real operational problems through a clear and practical approach, making it a valuable final year project with real-world relevance.